CLASS ASSIGNMENT 7
DATE : 08-10-2025
ID : 12341550

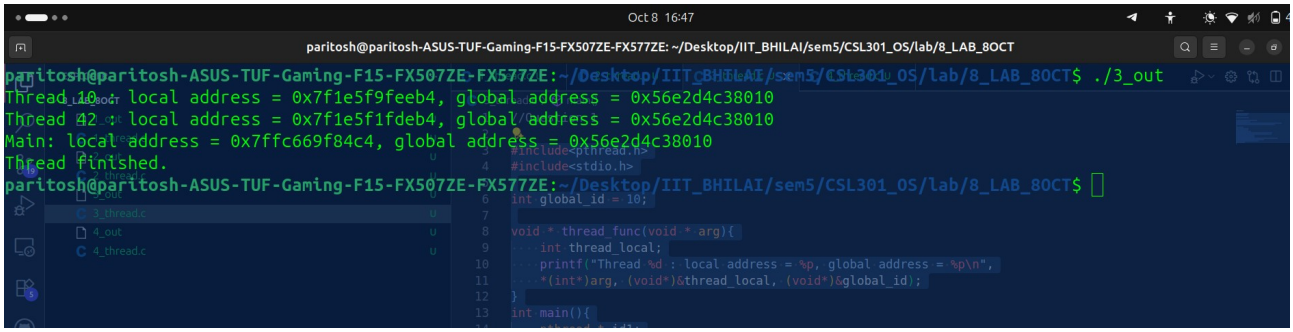
QUESTION 1

```
#include<pthread.h>
#include<stdio.h>

int global_id = 10;

void * thread_func(void * arg){
   int thread_local;
   printf("Thread %d : local address = %p, global address =
%p\n",
   *(int*)arg, (void*)&thread_local, (void*)&global_id);
}
int main(){
   pthread_t id1;
   pthread_t id2;
   int local_id = 42;
   pthread_create(&id1,NULL,thread_func,&global_id);
   pthread_create(&id2,NULL,thread_func,&local_id);
   pthread_join(id1,NULL);
   pthread_join(id2,NULL);
   printf("Main: local address = %p, global address = %p\
n",
   (void*)&local_id, (void*)&global_id);
   printf("Thread finished.\n");
   return 0;
}
```

OUTPUT :



EXPLANATION:
local address is different while global address remain same. I think local created a child thats why local addresses are different.

QUESTION 2

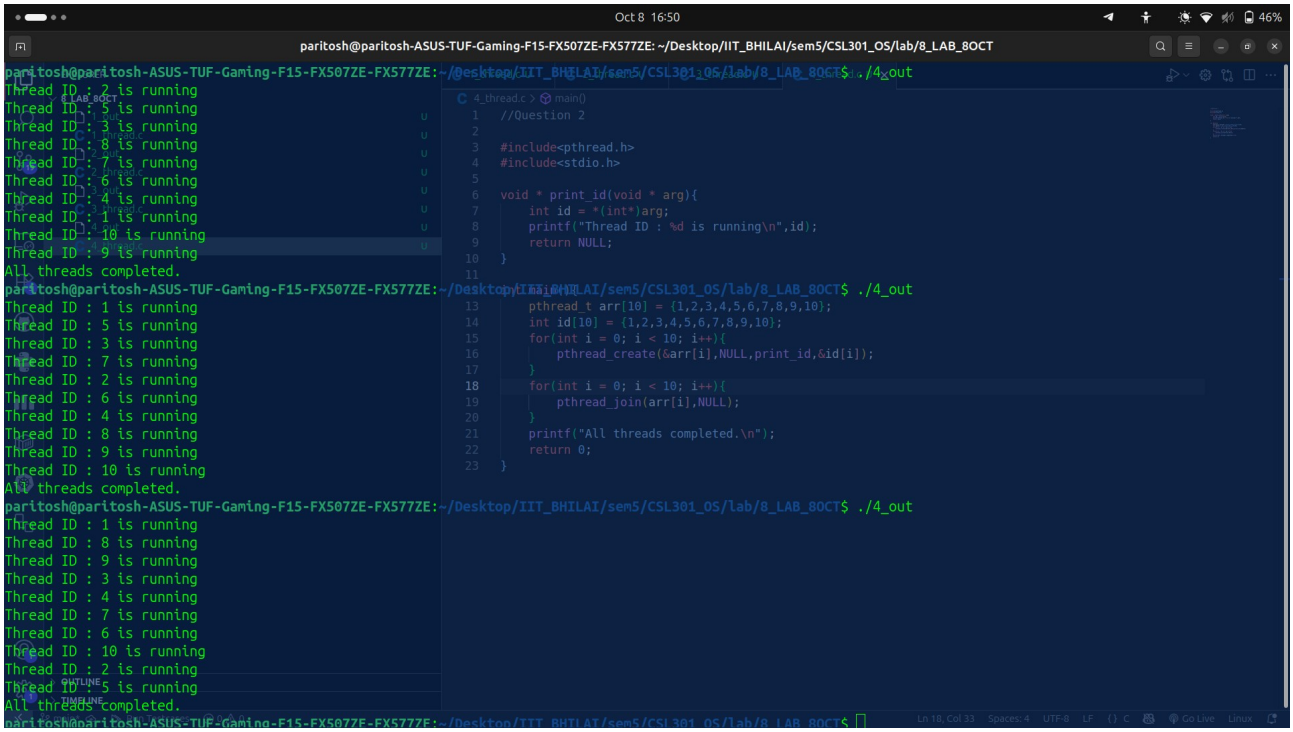
```
#include<pthread.h>
#include<stdio.h>

void * print_id(void * arg){
    int id = *(int*)arg;
    printf("Thread ID : %d is running\n",id);
    return NULL;
}

int main(){
    pthread_t arr[10] = {1,2,3,4,5,6,7,8,9,10};
    int id[10] = {1,2,3,4,5,6,7,8,9,10};
    for(int i = 0; i < 10; i++){
        pthread_create(&arr[i],NULL,print_id,&id[i]);
    }
    for(int i = 0; i < 10; i++){
        pthread_join(arr[i],NULL);
    }
```

```
    printf("All threads completed.\n");
    return 0;
}
```

OUTPUT :



EXPLANATION:

At each run ordering is not same because in threadinng ordering doesn’t matter.

QUESTION 3

```
#include<pthread.h>
#include<stdio.h>

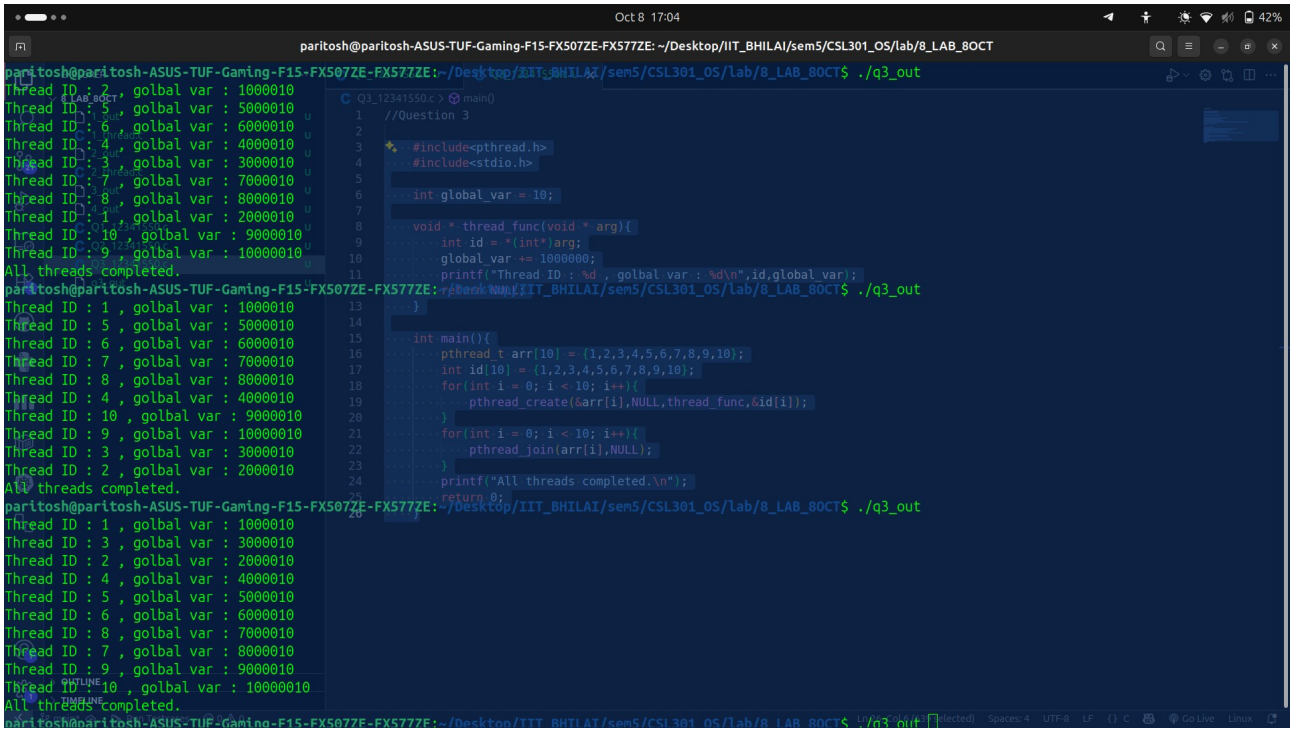
int global_var = 10;

void * thread_func(void * arg){
    int id = *(int*)arg;
```

```
    global_var += 1000000;
    printf("Thread ID : %d , golbal var : %d\n",id,global_var);
    return NULL;
}

int main(){
    pthread_t arr[10] = {1,2,3,4,5,6,7,8,9,10};
    int id[10] = {1,2,3,4,5,6,7,8,9,10};
    for(int i = 0; i < 10; i++){
        pthread_create(&arr[i],NULL,thread_func,&id[i]);
    }
    for(int i = 0; i < 10; i++){
        pthread_join(arr[i],NULL);
    }
    printf("All threads completed.\n");
    return 0;
}
```

OUTPUT :



EXPLANATION:

QUESTION 4

```
#include<pthread.h>
#include<stdio.h>
#include<stdlib.h>

void * compute_square(void * arg){
    int number = *(int*)arg;
    int * result = malloc(sizeof(int));
    *result = number * number;
    return result;
}

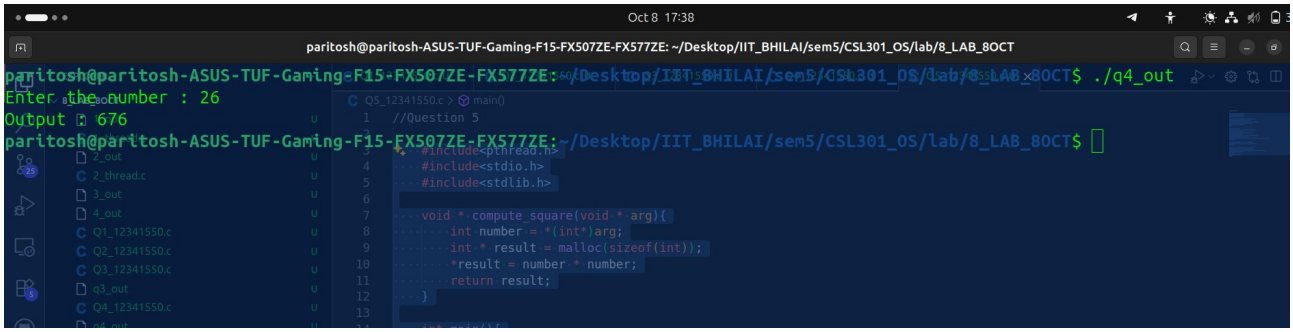
int main(){
    pthread_t tid;
    void * result;
    int number;
    printf("Enter the number : ");
    scanf("%d",&number);
    pthread_create(&tid,NULL,compute_square,&number);
    pthread_join(tid,&result);

    int * value = (int*)result;
    printf("Output : %d\n",*value);

    free(value);

    return 0;
}
```

OUTPUT :



EXPLANATION:

QUESTION 5

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

void * compute_square(void * arg){
    int number = *(int*)arg;
    int * result = malloc(sizeof(int));
    *result = number * number;
    return result;
}

int main(){
    int n;
    printf("Enter number of threads : ");
    scanf("%d", &n);

    pthread_t tid[n];
```

```
    void * res[n];
    int * nums = malloc(n * sizeof(int));

    for(int i = 0; i < n; i++){
        printf("Enter the number : ");
        scanf("%d", &nums[i]);
        pthread_create(&tid[i], NULL, compute_square,
&nums[i]);
    }

    for(int i = 0; i < n; i++){
        pthread_join(tid[i], &res[i]);
    }

    for(int i = 0; i < n; i++){
        int * val = (int*)res[i];
        printf("Thread %d returned %d\n", nums[i], *val);
        free(val);
    }

    free(nums);
    return 0;
}
```

OUPTPUT :

